

Contents

Topizzy Smart Contract Security Audit Report	1
Executive Summary	1
Protocol Summary	2
Findings	2
Test Suite Analysis	7
Post-Audit Recommendations	8
Airtime.sol — Audit Response	8
Response Summary	9
Detailed Findings and Responses	10
Informational Findings	17
Summary of Contract Changes	17
Post-Implementation	18

Topizzy Smart Contract Security Audit Report

Prepared by: Dennis Kiptoo — Security Researcher

Version: 1.0

Date: May 7, 2026

Contract audited: src/Airtime.sol

Commit hash: a20bbdfa7ac5cb4730592fe7dc799939f1998c02

Audit duration: 2 days | ~100 lines of code

Chain: Base (Ethereum L2)

Repository: <https://github.com/cableGraph/topizzy-security-audit/>

Executive Summary

12 total findings: 2 Critical, 3 High, 4 Medium, 3 Low — all Open.

Severity	Count	Status
Critical	2	Open
High	3	Open
Medium	4	Open
Low	3	Open

The root cause underlying both Critical findings is the **complete absence of on-chain deposit accounting**. The contract is a pure escrow with no record of who deposited what amount, meaning all solvency guarantees depend entirely on the off-chain backend behaving correctly.

Protocol Summary

Topizzy is an onchain airtime purchase tool on Base, allowing users to top up mobile airtime using USDC without off-ramping. A flat fee of \$0.05 applies per transaction, with automatic refunds for failed top-ups. Supported regions: Kenya, Uganda, Rwanda, Tanzania, South Africa (in progress).

Core escrow pattern:

User --deposit USDC--> [Airtime.sol] --treasury controls--> Treasury wallet

Critical business invariants: 1. Every deposited USDC must be withdrawn by treasury OR refunded to the original depositor. 2. A failed airtime order MUST result in a full refund. 3. No user should be refunded more than they deposited. 4. The \$0.05 flat fee must be extractable by treasury. 5. No user should be able to drain another user's funds.

Findings

CRITICAL

[C-01] No on-chain refund accounting — double refund drains protocol

Severity: Critical | **Likelihood:** High | **Impact:** High | **Status:** Open

Description: The refund() function transfers USDC to a receiver with no on-chain verification that orderRef has not been previously refunded. The contract stores no per-order deposit records. Treasury can call refund() multiple times with the same orderRef, and each call processes independently as long as the contract holds sufficient balance. There is also no upper bound linking amount to the original deposit — treasury can refund Alice 1000 USDC even if she only deposited 10 USDC, consuming other depositors' funds.

Impact: A single duplicate refund call or over-refund drains other depositors' funds. Users whose airtime failed cannot be refunded if their funds were consumed by a prior over-refund. No on-chain audit trail to detect or recover from this state.

Proof of Concept (confirmed, gas: 141,036):

```
// Alice deposits 10 USDC
airtime.deposit("order-42", TEN);
// Treasury refunds order-42 legitimately
airtime.refund("order-42", alice, TEN);
// Treasury refunds order-42 AGAIN — SUCCEEDS
airtime.refund("order-42", alice, TEN);
// Alice received 20 USDC from a 10 USDC deposit
```

Also confirmed via fuzz testing: testFuzz_C01_RefundCanExceedDeposit across 256 random pairs. Invariant invariant_TotalRefundedNeverExceedsTotalDeposited was

immediately broken when a `refundExistingOrder()` handler was added.

Recommended Fix:

```
mapping(bytes32 => bool)    private s_refundedOrders;
mapping(bytes32 => uint256) private s_orderAmounts;

// In deposit(): record order amount
bytes32 orderHash = keccak256(abi.encodePacked(depositRef, msg.sender));
s_orderAmounts[orderHash] = amount;

// In refund(): add before transfer
bytes32 orderHash = keccak256(abi.encodePacked(orderRef, receiver));
require(!s_refundedOrders[orderHash], "Order already refunded");
require(amount <= s_orderAmounts[orderHash], "Refund exceeds original deposit");
s_refundedOrders[orderHash] = true;
```

[C-02] Treasury withdrawal has no minimum balance check — refund liquidity can be drained **Severity:** Critical | **Likelihood:** High | **Impact:** High | **Status:** Open

Description: `withdrawTreasury()` has no minimum balance check and no awareness of pending refund obligations. Treasury can withdraw 100% of the contract's USDC balance at any time — including funds belonging to users with pending orders. If orders then fail, there is no USDC left to refund users.

Impact: Users lose deposited funds permanently with no on-chain recourse. This is an operational race condition between withdrawal and refund processing — no malicious code required.

Proof of Concept (confirmed, gas: 163,428):

```
// Alice and Bob each deposit 10 USDC (contract holds 20 USDC)
// Treasury withdraws everything
airtime.withdrawTreasury(treasury, 20e6);
// Airtime fails. Treasury tries to refund Alice.
airtime.refund("order-A", alice, TEN); // REVERTS — no balance
// Alice lost 10 USDC with no on-chain recourse
```

Recommended Fix:

```
uint256 public pendingRefunds;

function setPendingRefunds(uint256 amount) external onlyTreasury {
    pendingRefunds = amount;
}

function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury nonRe
    uint256 available = IERC20(usdcToken).balanceOf(address(this)) - pendingRefunds;
```

```
require(amount <= available, "Cannot withdraw pending refund liquidity");
IERC20(usdcToken).safeTransfer(receiver, amount);
emit TreasuryWithdrawal(receiver, amount);
}
```

HIGH

[H-01] No treasury rotation mechanism — key compromise is irrecoverable

Severity: High | **Likelihood:** Medium | **Impact:** High | **Status:** Open

Description: treasury is set once in the constructor as public immutable. No set-Treasury(), proposeTreasury(), or ownership transfer function exists. A compromised treasury key gives an attacker permanent, irrevocable full control over all user funds with no on-chain recovery path. The team cannot revoke access, pause the treasury role, or migrate without redeploying.

Proof of Concept (confirmed, gas: 101,581): Calling setTreasury(address) returns false (function does not exist). A prank as the attacker-controlled treasury successfully drains the contract.

Recommended Fix: Implement two-step treasury transfer with a 2-day delay:

```
address public treasury;
address public pendingTreasury;
uint256 public treasuryTransferTime;
uint256 public constant TREASURY_DELAY = 2 days;

function proposeTreasury(address newTreasury) external onlyTreasury { ... }
function acceptTreasury() external {
    require(msg.sender == pendingTreasury, "Not pending treasury");
    require(block.timestamp >= treasuryTransferTime, "Delay not passed");
    treasury = pendingTreasury;
    pendingTreasury = address(0);
}
```

[H-02] depositWithPermit() vulnerable to permit front-run griefing DOS

Severity: High | **Likelihood:** Medium | **Impact:** High | **Status:** Open

Description: depositWithPermit() calls permit() using a user-supplied signature. An attacker monitoring the mempool can extract the signature and call permit() directly first, consuming the nonce. When the user's transaction executes, permit() reverts with a stale nonce error, reverting the entire deposit. Users pay gas for a guaranteed failure.

Proof of Concept (confirmed, gas: 134,532): Attacker front-runs user's permit() call; user's subsequent depositWithPermit() reverts with ERC2612InvalidSigner.

Recommended Fix: Wrap permit() in a try/catch and fall through to existing allowance:

```
try IERC20Permit(usdcToken).permit(msg.sender, address(this), amount, deadline, v, r, s)
{} catch {
    uint256 currentAllowance = IERC20(usdcToken).allowance(msg.sender, address(this));
    require(currentAllowance >= amount, "Permit failed and insufficient allowance");
}
IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
```

[H-03] Zero per-user deposit accounting — entire protocol solvency is off-chain Severity: High | Likelihood: High | Impact: Medium | Status: Open

Description: The contract stores no per-user or per-order deposit data (mapping(address => uint256), mapping(bytes32 => uint256), etc.). It cannot verify on-chain that a refund amount matches what was deposited, who originally paid, or whether an order has been settled. The global depositCounter is never linked to a user or amount.

Proof of Concept (confirmed, gas: 105,328): Treasury can call refund("order-A", attacker, TEN) even though Alice paid for order-A — contract accepts it, attacker receives Alice's funds.

Recommended Fix:

```
struct OrderRecord { address payer; uint256 amount; bool settled; }
mapping(bytes32 => OrderRecord) public orders;

function deposit(string memory depositRef, uint256 amount) external nonReentrant {
    bytes32 orderHash = keccak256(abi.encodePacked(depositRef, msg.sender));
    require(orders[orderHash].amount == 0, "Order already exists");
    IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
    orders[orderHash] = OrderRecord({ payer: msg.sender, amount: amount, settled: false,
    ...
}
```

MEDIUM

[M-01] withdrawTreasury() missing nonReentrant modifier Severity: Medium | Likelihood: Low | Impact: High | Status: Open

refund() has nonReentrant; withdrawTreasury() performs an identical operation but does not. Low risk with USDC on Base (no reentrancy hooks), but elevated risk if the

token is ever changed. **Fix:** Add nonReentrant to withdrawTreasury().

[M-02] No minimum deposit amount — dust grieving and fee bypass **Severity:** Medium | **Likelihood:** Medium | **Impact:** Medium | **Status:** Open

Contract accepts any deposit > 0. Attacker can spam 1 wei deposits, each requiring backend API calls, off-chain processing, and a treasury refund transaction. The documented \$0.05 flat fee has no on-chain enforcement. **Fix:**

```
uint256 public constant MIN_DEPOSIT = 100_000; // $0.10 USDC (6 decimals)
require(amount >= MIN_DEPOSIT, "Below minimum deposit");
```

[M-03] No validation that usdcToken supports EIP-2612 permit **Severity:** Medium | **Likelihood:** Low | **Impact:** High | **Status:** Open

Constructor accepts any non-zero address as _ERC20TokenAddress. If the token does not implement EIP-2612, depositWithPermit() either silently succeeds (with a fallback function, potentially with zero allowance) or permanently reverts. Since usdcToken is immutable, this cannot be corrected post-deployment. **Fix:** Add IERC20Metadata(_ERC20TokenAddress).decimals() > 0 validation in constructor.

[M-04] Unconstrained string length in depositRef / orderRef **Severity:** Medium | **Likelihood:** Low | **Impact:** Medium | **Status:** Open

No length validation on depositRef/orderRef. Attacker can submit 100,000-byte strings: bloating chain event logs, causing backend parsing errors, SQL injection if strings are interpolated into database queries, and log injection. **Fix:**

```
uint256 public constant MAX_REF_LENGTH = 64;
require(bytes(ref).length <= MAX_REF_LENGTH, "Reference too long");
```

LOW

[L-01] depositCounter provides no security value Never stored against a user, amount, or order. Cannot be queried for meaningful state. Costs ~20,000 gas (cold SSTORE) per deposit for zero security benefit. **Fix:** Remove entirely, or replace with meaningful per-order tracking (H-03 fix).

[L-02] refund() and withdrawTreasury() are functionally redundant Both perform identical safeTransfer with identical onlyTreasury access control. refund("drain", treasury, entireBalance) is functionally equivalent to withdrawTreasury(treasury, entireBalance). **Fix:** Consolidate into a single internal _transferFromContract() function.

[L-03] USDC centralization / blacklist dependency undocumented Circle can blacklist any address or pause USDC globally. If Airtime.sol is blacklisted, all deposits, refunds, and withdrawals revert permanently — all held funds become inaccessible. **Fix:** Document this risk; consider supporting a secondary stablecoin; make usdcToken mutable (with proper access control) for emergency migration.

Informational

- **[I-01]** Emit depositId return value in the OrderPaid event for easier off-chain reconciliation.
- **[I-02]** Verify treasury is declared immutable in the final deployed version.
- **[I-03]** depositCounter = 0 in constructor is redundant — Solidity zero-initialises by default.
- **[I-04]** Consider an emergency Paused mechanism even before the H-01 treasury rotation fix is implemented.
- **[I-05]** Comment // ERC20 token address (USDC/USDT) suggests USDT was considered — note USDT does not revert on failed transfers on some chains (though SafeERC20 handles this).

Test Suite Analysis

19 tests total | 50,000+ state transitions | 19/19 passed

Type	File	Count	Runs
Unit PoC	test/unit/AirtimeUnit.t.sol	1	1 each
Fuzz	test/fuzz/AirtimeFuzz.t.sol	256	256 each
Invariant	test/fuzz/AirtimeInvariants.t.sol	1000	1000 runs × 50 depth

Invariant handler executed: 16,647 deposits, 16,643 refunds, 16,710 withdrawals.

Key finding from invariant suite: invariant_TotalRefundedNeverExceedsTotalDeposited **broke immediately** when a refundExistingOrder() handler was added — Foundry produced the exact sequence deposit("order-X", 10e6) → refund("order-X", user, 10e6) → refund("order-X", user, 10e6) proving C-01 mechanically.

Gas snapshot:

Test	Gas
testDepositWithPermit	122,938
test_C01_DoubleRefund_DrainsFunds	139,220
test_C01b_RefundExceedsDeposit	146,946
test_C02_WithdrawBeforeRefund_BlocksUsers	161,611
test_H01_NoTreasuryRotation_IsPermanent	99,872
test_H02_PermitFrontrun_RevertsUserDeposit	134,532
test_H03_NoAccounting_WrongUserRefunded	103,620
test_M01_WithdrawTreasury_MissingReentrancyGuard	4,477
test_M02_DustDeposit_NoMinimumEnforced	224,098

Slither static analysis confirmed M-01, M-02, and M-04. It did NOT detect C-01, C-02, H-01, H-02, or H-03 — all Critical and High findings required manual review.

Post-Audit Recommendations

Immediate (before any mainnet deployment): 1. Fix C-01 — implement on-chain per-order accounting. 2. Fix C-02 — enforce minimum balance before treasury withdrawals. 3. Fix H-02 — add try/catch around permit() in depositWithPermit().

Before launch: 4. Fix H-01 — two-step treasury rotation with 48-hour delay. 5. Fix M-01 — add nonReentrant to withdrawTreasury(). 6. Fix M-02 — enforce MIN_DEPOSIT on-chain. 7. Fix M-04 — enforce maximum string length on refs.

Architecture recommendations: - Replace single EOA treasury with a Gnosis Safe (2-of-3 minimum). - Add a pause() circuit breaker callable by a separate guardian address. - Keep the refundExistingOrder() handler in invariant tests permanently as a regression check for C-01. - A mitigation review is strongly recommended after fixes are implemented.

Auditor: Dennis Kiptoo | **Date:** May 7, 2026 | **Status:** Awaiting client response on all findings

Airtime.sol — Audit Response

Audit by: Dennis Kiptoo — May 7, 2026

Response by: Topizzy — May 2026

Branch: audit/smart-contract-fixes

Repo: <https://github.com/cableGraph/topizzy-security-audit/>

Contract: src/Airtime.sol

Chain: Base (Ethereum L2)

Response Summary

ID	Auditor Severity	Auditor Finding	Topizzy Response	Status
C-01	Critical	No on-chain refund accounting — double refund drains protocol	Implementing: OrderRecord mapping with settled flag and amount cap	Fixing
C-02	Critical	Treasury withdrawal has no minimum balance check	Gnosis Safe 2-of-3 multisig — withdrawals require 2 signers. Single key cannot drain contract. Off-chain pending balance guard in withdrawal UI	Operational fix
H-01	High	No treasury rotation mechanism — key compromise is irrecoverable	Not implementing. Treasury is immutable Gnosis Safe address — signer rotation handled within the Safe, not the contract	Not implementing
H-02	High	depositWithPermit() vulnerable to permit front-run griefing	Implementing: try/catch fallback to existing allowance	Fixing
H-03	High	Zero per-user deposit accounting — solvency is off-chain	Closed by C-01 fix — OrderRecord ties each order to its payer	Closed via C-01
M-01	Medium	withdrawTreasury() missing nonReentrant	Implementing: one-line addition	Fixing
M-02	Medium	No minimum deposit — dust spam and fee bypass	Auditor's framing does not apply to our architecture. Policy: direct deposits are donations	Not implementing
M-03	Medium	No validation that usdcToken supports EIP-2612	Implementing: constructor calls DOMAIN_SEPARATOR() at deploy time	Fixing

ID	Auditor Severity	Auditor Finding	Topizzy Response	Status
M-04	Medium	Unconstrained string length in depositRef / orderRef	Implementing: MAX_REF_LENGTH = 16 — backend uses nanoid(8), 16 gives a small buffer	Fixing
L-01	Low	depositCounter provides no security value	Implementing: removing it, replaced by OrderRecord mapping	Fixing
L-02	Low	refund() and withdrawTreasury() are functionally redundant	Not consolidating — semantically distinct, clarity matters more than brevity	Not implementing
L-03	Low	USDC centralization / blacklist dependency undocumented	Acknowledging risk in NatSpec. usdcToken stays immutable — mutability adds more risk than it removes	Documenting only

Detailed Findings and Responses

[C-01] No on-chain refund accounting — double refund drains protocol

Auditor Severity: Critical | **Likelihood:** High | **Impact:** High

Auditor's Finding: The refund() function transfers USDC to a receiver with no on-chain verification that orderRef has not been previously refunded. The contract stores no per-order deposit records. Treasury can call refund() multiple times with the same orderRef, and each call processes independently as long as the contract holds sufficient balance. There is also no upper bound linking amount to the original deposit — treasury can refund Alice 1000 USDC even if she only deposited 10 USDC, consuming other depositors' funds.

Auditor's Proof of Concept (confirmed, gas: 141,036):

```
airtime.deposit("order-42", TEN);
airtime.refund("order-42", alice, TEN); // legitimate
airtime.refund("order-42", alice, TEN); // succeeds — Alice received 20 USDC from a 1
```

Auditor's Recommended Fix:

```

mapping(bytes32 => bool)    private s_refundedOrders;
mapping(bytes32 => uint256) private s_orderAmounts;

bytes32 orderHash = keccak256(abi.encodePacked(orderRef, receiver));
require(!s_refundedOrders[orderHash], "Order already refunded");
require(amount <= s_orderAmounts[orderHash], "Refund exceeds original deposit");
s_refundedOrders[orderHash] = true;

```

Topizzy Response: Agreed. Implementing with a slightly richer structure than the auditor's recommendation — a single OrderRecord struct instead of two separate mappings, and adding the payer address so the contract can verify the refund recipient matches who originally deposited.

```

struct OrderRecord {
    address payer;
    uint256 amount;
    bool    settled;
}
mapping(bytes32 => OrderRecord) public orders;

```

Key is keccak256(abi.encodePacked(depositRef, msg.sender)) on deposit, and the same hash on refund (using receiver as the second argument, which must match the stored payer). This also closes H-03 (see below).

Gas impact: One cold SSTORE on deposit (user pays). One SSTORE update on refund (treasury pays — already paying for this tx). Zero overhead on the success/withdrawal path.

[C-02] Treasury withdrawal has no minimum balance check — refund liquidity can be drained

Auditor Severity: Critical | **Likelihood:** High | **Impact:** High

Auditor's Finding: withdrawTreasury() has no minimum balance check and no awareness of pending refund obligations. Treasury can withdraw 100% of the contract's USDC balance at any time — including funds belonging to users with pending orders. If orders then fail, there is no USDC left to refund users.

Auditor's Proof of Concept (confirmed, gas: 163,428):

```

airtime.deposit("order-A", TEN);
airtime.deposit("order-B", TEN);           // contract holds 20 USDC
airtime.withdrawTreasury(treasury, 20e6); // treasury takes everything
airtime.refund("order-A", alice, TEN);     // REVERTS – no balance. Alice lost 10 USDC.

```

Auditor's Recommended Fix:

```

uint256 public pendingRefunds;
function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury nonRe
    uint256 available = IERC20(usdcToken).balanceOf(address(this)) - pendingRefunds;
    require(amount <= available, "Cannot withdraw pending refund liquidity");
    ...
}

```

Topizzy Response: The auditor's recommended on-chain fix requires `settleOrder()` to be called after every successful airtime delivery to keep `pendingRefunds` accurate. At ~\$0.061 profit per Kenya transaction (our primary market), an extra treasury gas call per successful order erodes ~1.6% of margin. The happy path must remain gas-free for the treasury.

We are not implementing an on-chain balance guard. Instead, all `withdrawTreasury()` calls will require **Gnosis Safe 2-of-3 multisig approval**:

Gnosis Safe setup (before mainnet): - Deploy a Gnosis Safe on Base with 3 signers and a threshold of 2 - Recommended signer composition: 1 hardware wallet (Ledger/Trezor) + 2 team EOAs, no two on the same device - Set the Safe address as `_treasury` at contract deployment - Every `withdrawTreasury()` call must be queued in the Safe UI and approved by 2-of-3 signers before it executes on-chain — a single compromised key cannot drain the contract unilaterally

Supporting operational controls: - Backend maintains a running `pendingBalance` (sum of deposits with no resolved outcome yet). Treasury withdrawal UI shows `contractBalance - pendingBalance` as the safe withdrawal amount - The C-01 fix caps each refund to its original deposit amount, bounding the impact of any over-refund bug

The auditor's concern is valid. Our mitigation is the Gnosis Safe threshold requirement rather than an on-chain balance guard — this addresses the higher-risk scenario (single key compromise) more directly than the auditor's recommended fix.

[H-01] No treasury rotation mechanism — key compromise is irrecoverable

Auditor Severity: High | **Likelihood:** Medium | **Impact:** High

Auditor's Finding: treasury is set once in the constructor as public immutable. No `setTreasury()`, `proposeTreasury()`, or ownership transfer function exists. A compromised treasury key gives an attacker permanent, irrevocable full control over all user funds with no on-chain recovery path. The team cannot revoke access, pause the treasury role, or migrate without redeploying.

Auditor's Recommended Fix: Two-step treasury transfer with a 2-day delay:

```

address public treasury;
address public pendingTreasury;
uint256 public treasuryTransferTime;

```

```

uint256 public constant TREASURY_DELAY = 2 days;

function proposeTreasury(address newTreasury) external onlyTreasury { ... }
function acceptTreasury() external {
    require(msg.sender == pendingTreasury, "Not pending treasury");
    require(block.timestamp >= treasuryTransferTime, "Delay not passed");
    treasury = pendingTreasury;
    pendingTreasury = address(0);
}

```

Topizzy Response: Not implementing the rotation mechanism. Reason: treasury will be set to a **Gnosis Safe address at deployment and kept immutable**. Signer rotation (e.g. a team member leaving) is handled within the Safe itself — owners can be added or removed, and the threshold can be changed, all without touching the contract. The Safe address stays constant.

This makes the auditor’s concern moot: the equivalent of “key rotation” happens at the Safe layer, not the contract layer. Adding proposeTreasury/acceptTreasury would only be necessary if the Safe address itself needed to change, which would require redeployment regardless given the Safe provides stronger guarantees than an on-chain timelock.

treasury remains immutable. No rotation functions added.

[H-02] depositWithPermit() vulnerable to permit front-run griefing DOS

Auditor Severity: High | **Likelihood:** Medium | **Impact:** High

Auditor’s Finding: depositWithPermit() calls permit() using a user-supplied signature. An attacker monitoring the mempool can extract the signature and call permit() directly first, consuming the nonce. When the user’s transaction executes, permit() reverts with a stale nonce error, reverting the entire deposit. Users pay gas for a guaranteed failure.

Auditor’s Recommended Fix:

```

try IERC20Permit(usdcToken).permit(msg.sender, address(this), amount, deadline, v, r, s)
{} catch {
    uint256 currentAllowance = IERC20(usdcToken).allowance(msg.sender, address(this));
    require(currentAllowance >= amount, "Permit failed and insufficient allowance");
}
IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);

```

Topizzy Response: Agreed. Implementing exactly as recommended. If the bot front-ran the permit, the allowance is already set (the permit succeeded from the bot’s

call), so `safeTransferFrom` proceeds normally. The attacker wasted gas for nothing; the user's deposit goes through.

[H-03] Zero per-user deposit accounting — entire protocol solvency is off-chain

Auditor Severity: High | **Likelihood:** High | **Impact:** Medium

Auditor's Finding: The contract stores no per-user or per-order deposit data. It cannot verify on-chain that a refund amount matches what was deposited, who originally paid, or whether an order has been settled. Treasury can call `refund("order-A", attacker, TEN)` even though Alice paid for order-A.

Auditor's Recommended Fix:

```
struct OrderRecord { address payer; uint256 amount; bool settled; }
mapping(bytes32 => OrderRecord) public orders;
```

Topizzy Response: Closed by the C-01 fix. The `OrderRecord` struct stores payer, amount, and settled per order. `refund()` validates that receiver matches the stored payer, that the order is not already settled, and that amount does not exceed the original deposit. No additional changes needed beyond C-01.

[M-01] `withdrawTreasury()` missing `nonReentrant` modifier

Auditor Severity: Medium | **Likelihood:** Low | **Impact:** High

Auditor's Finding: `refund()` has `nonReentrant`; `withdrawTreasury()` performs an identical token transfer but does not. Low risk with USDC on Base today (no reentrancy hooks), but elevated risk if the token is ever changed.

Topizzy Response: Agreed. Adding `nonReentrant` to `withdrawTreasury()`. One-line change, no reason not to.

[M-02] No minimum deposit — dust spam and fee bypass

Auditor Severity: Medium | **Likelihood:** Medium | **Impact:** Medium

Auditor's Finding: Contract accepts any deposit > 0 . Attacker can spam 1 wei deposits, each requiring backend API calls, off-chain processing, and a treasury refund transaction. The documented \$0.05 flat fee has no on-chain enforcement.

Topizzy Response: The auditor's threat model does not apply to our architecture. A deposit to the smart contract does not trigger airtime delivery — the backend drives the entire user interaction and holds the phone number. The flow is:

1. User provides phone number and amount to the backend
2. Backend creates an order and generates a depositRef
3. User deposits via the contract using that ref
4. Backend matches the OrderPaid event to the pending order and calls Africa's Talking

A deposit made directly to the contract with no backend-generated ref will never trigger an AT API call — the backend has no phone number for it.

Policy decision: Direct deposits made outside the backend flow are treated as voluntary donations. Treasury has no obligation to refund them unless the depositor contacts Topizzy directly. If a refund is agreed, the gas cost of the refund() call must be covered out-of-band by the depositor — treasury will not absorb gas costs for unsolicited deposits.

No contract change for this finding.

[M-03] No validation that usdcToken supports EIP-2612 permit

Auditor Severity: Medium | **Likelihood:** Low | **Impact:** High

Auditor's Finding: Constructor accepts any non-zero address as _ERC20TokenAddress. If the token does not implement EIP-2612, depositWithPermit() either silently succeeds with zero allowance or permanently reverts. Since usdcToken is immutable, this cannot be corrected post-deployment.

Topizzy Response: Agreed. Adding a deploy-time check in the constructor:

```
IERC20Permit(_ERC20TokenAddress).DOMAIN_SEPARATOR();
```

If the token does not implement EIP-2612 this reverts at deployment, not at first user transaction. Does not catch a token that returns a junk domain separator, but catches the common mistake of passing a plain ERC20 address.

[M-04] Unconstrained string length in depositRef / orderRef

Auditor Severity: Medium | **Likelihood:** Low | **Impact:** Medium

Auditor's Finding: No length validation on depositRef/orderRef. Attacker can submit 100,000-byte strings: bloating chain event logs, causing backend parsing errors, SQL injection if strings are interpolated into database queries, and log injection.

Topizzy Response: Agreed. The backend generates orderRef using nanoid(8) — exactly 8 URL-safe characters (A-Za-z0-9_-). Setting the limit to 16 to match actual usage with a small buffer for future format changes:

```
uint256 public constant MAX_REF_LENGTH = 16;  
require(bytes(depositRef).length <= MAX_REF_LENGTH, "Reference too long");
```

Applied in deposit(), depositWithPermit(), and refund(). Any ref longer than 16 characters was never generated by our backend and is either a mistake or an attack.

[L-01] depositCounter provides no security value

Auditor's Finding: Never stored against a user, amount, or order. Cannot be queried for meaningful state. Costs ~20,000 gas (cold SSTORE) per deposit for zero security benefit.

Topizzy Response: Agreed. Removing depositCounter entirely. The OrderRecord mapping introduced by the C-01 fix replaces it with per-order data that is actually queryable and meaningful.

[L-02] refund() and withdrawTreasury() are functionally redundant

Auditor's Finding: Both perform identical safeTransfer with identical onlyTreasury access control. Suggested fix: consolidate into a single internal _transferFromContract() function.

Topizzy Response: Not implementing. The two functions are semantically distinct — refund() returns a user's funds when their order failed; withdrawTreasury() collects settled revenue. Keeping them separate makes the contract's intent readable at a glance and makes event log analysis straightforward. The implementation similarity is not a problem.

[L-03] USDC centralization / blacklist dependency undocumented

Auditor's Finding: Circle can blacklist any address or pause USDC globally. If Airtime.sol is blacklisted, all deposits, refunds, and withdrawals revert permanently — all held funds become inaccessible.

Topizzy Response: Risk acknowledged. We are not making usdcToken mutable — adding mutability introduces a new attack surface (whoever can change the token address can redirect all deposits to a malicious contract). The risk is documented in the

contract NatSpec. Mitigation: keep contract balance low by withdrawing frequently; treasury operates as a Gnosis Safe so blacklist response can be coordinated across signers.

Informational Findings

ID	Auditor's Note	Topizzy Response
I-01	Emit depositId in OrderPaid event for easier off-chain reconciliation	Implementing — emitting orderHash (the bytes32 key) in OrderPaid instead of a counter. Gives backend a direct on-chain handle for each order
I-02	Verify treasury is declared immutable in final deployed version	Confirmed immutable — Gnosis Safe address is set at deployment and never changes. Signer rotation is managed within the Safe
I-03	depositCounter = 0 in constructor is redundant	Moot — depositCounter is being removed entirely (L-01)
I-04	Consider an emergency Paused mechanism	Pending internal team discussion — specifically who holds the pauser role. To be revisited before mainnet
I-05	USDT comment suggests USDT was considered — note USDT doesn't revert on failed transfers	Contract is USDC-only. The comment // ERC20 token address (USDC/USDT) will be updated to // USDC token address (Base: 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913)

Summary of Contract Changes

#	Change	Finding
1	Remove depositCounter, add OrderRecord struct + orders mapping	C-01, H-03, L-01
2	Add orderHash to OrderPaid event	I-01
3	Add MAX_REF_LENGTH constant	M-04
4	Add DOMAIN_SEPARATOR() check in constructor	M-03
5	Update usdcToken comment to USDC-only with Base contract address	I-05

#	Change	Finding
6	Add MAX_REF_LENGTH check + OrderRecord write in deposit()	C-01, M-04
7	Add MAX_REF_LENGTH check + try/catch permit + OrderRecord write in depositWithPermit()	C-01, H-02, M-04
8	Add settled check, payer check, amount cap + mark settled in refund()	C-01
9	Add nonReentrant to withdrawTreasury()	M-01

Post-Implementation

- Update test/ to cover: double refund reverts, over-refund reverts, wrong-payer refund reverts, permit front-run fallthrough, treasury rotation flow, long ref revert.
- Run forge test and forge snapshot to confirm gas deltas.
- Move treasury to Gnosis Safe 2-of-3 before mainnet (C-02 operational fix).
- Request mitigation review from auditor (Dennis Kiptoo) once implemented.